CEMS 2.0
项目推进操作手册

如何推进落实项目，你要怎么做

这本手册只回答一个问题：蓝图已经有了，接下来每一步谁做、做什么、做到什么算过关。50 个功能、7 个阶段、三线并行约 14-16 周主线工期，全部拆成可执行、可验收、可回滚的动作。你不需要懂全部技术细节，你需要守住三件事：**拍板、验收、踩刹车**。

50	7	14-16 周	3 件
功能 F1-F50	阶段 Phase 0-6	三线并行主线工期	P0 地基先行

适用对象	项目拍板人（你本人）为主，执行与配合人员按章节查阅
依据文件	《06 优化建议（战略版）》《08 后端复刻方案 v2》《15 开源功能借鉴与开发改进计划》及项目交付规范
使用方式	开工前读第一、二章；每周按第三章推进；部署看第四章；每个功能按第六、七章交付；决策查第十章；出问题时查第十一章

> **先记住总顺序：**地基三件（安全、发票真实性、金额口径）→ 自动化基建 → 安全与断环 → 效率工具 → 可视化与可观测 → 体验补齐 → 集成预留 → 新赛道单独立项。顺序反了，后面全是返工。

目录与使用说明

章节	标题	你花的时间
第一章	决策闸门 0：不过闸门不开工	拍板 30 分钟
第二章	现在就能做的动作：第 1 周清单	本周执行
第三章	分阶段操作卡：Phase 0-6	每阶段开工前读
第四章	部署方案：Docker Compose 定案	搭环境时读
第五章	项目加速八法（含续：效率工具四件）	全程照做
第六章	通用执行 SOP：每个功能的 8 步	全程照做
第七章	交付规范：唯一发布通道与验收证据	每次上线必查
第八章	周节奏与沟通模板	每周固定动作
第九章	角色分工表	定人时查
第十章	决策清单：9 项已拍板（2026-10-19）	查结论时读
第十一章	风险红线：触碰即停	立项前必读
附录	可填写清单：启动单、阶段验收单、单功能验收单；附录 D Docker 验证结论备忘	打印填写

三种角色，全书统一口径

你拍板

只有你能定的事：花不花钱、改不改范围、上不上线、暂缓还是继续。这类事项在书里标为「拍板」。没有你的明确放行，执行方不得自行扩大范围。

助理执行

调研、写代码、部署、真机验证、出证据报告，都属于执行。执行方交付的不是「我改好了」，而是「服务器上验证通过，证据在这里」。

需外部配合：涉及公司制度、法务、财务口径、持牌服务商、业务部门试用的事项，单靠技术做不了。这类事项必须先拿到外部结论，再排期，不要先开发后补手续。

> **一条铁律：**原 CEMS 系统只做只读查看，不新建、不改、不删、不审批。所有开发、试验和上线，都在复刻工程和自有后端上进行。

怎么判断一件事做完了

1. 功能在服务器上真实可用，不是本地演示可用。
2. 有验收证据：截图、测试结果、监控或日志，任选其一但必须有。
3. 验收标准逐条打勾，写进附录的验收单。
4. 代码、文档、部署记录三处同步，缺一处不算交付。

第一章　决策闸门 0：不过闸门不开工

闸门 0 不是仪式，是省钱的关卡。过去的问题很清楚：手工部署导致过白屏，假数据能进账，金额对不上。闸门就是把这些坑在开工前堵住。四项全部打勾，才允许进入 Phase 0。

闸门项	具体标准	谁确认
G1 地基三件有着落	HTTPS 与证书来源已定；发票识别「接真服务」或「先禁用保存」二选一已定；金额口径由财务确认统一规则。三件各有负责人和日期。	你拍板
G2 仓库与通道就绪	私有仓库为唯一代码归宿；唯一发布通道已确认；禁止手工覆盖线上目录已写入团队约定。	助理执行
G3 后端四项已放行	技术栈、数据库、启动范围、部署位置四项，按第十章已拍板结论执行。未放行，后端不开工。	你拍板
G4 每阶段有验收人	每个阶段写明验收人和验收标准来源。没有验收人的阶段，不排期。	你指定

> **过闸动作（30 分钟）：**打开第十章核对已拍板结论。九项决策已于 2026-10-19 全部按默认通过，闸门 0 的拍板环节已经完成，剩下的是把结论落进台账，等开工令下达即可启动。

第二章　现在就能做的动作：第 1 周清单

第一周不写大功能，只做七件小而关键的事。每件都有明确产出，做完一件勾一件。

#	动作	产出物	负责
1	**后端四项已拍板（2026-10-19）**：Spring Boot 3.2、MySQL 8.0、Phase 0+1 先行、同机部署，结论见第十章，本周落进台账。	台账记录	你拍板
2	**给发票识别定性**：接真实识别服务，或先禁用保存入口。二选一，不允许「先放着」。	书面结论	你拍板
3	**启动 HTTPS 排期**：确认域名、证书、切换窗口。它同时是离线应用的前置条件。	排期表	助理执行
4	**清理默认账号风险**：强制改默认口令，登录页不再展示默认账号。	整改截图	助理执行
5	**冻结金额必填规则**：合同、发票、预算、工资等关键金额字段改为必填，应发与实发改为自动求和只读，先定规则后改代码。	字段规则表	需财务配合
6	**确认唯一发布通道**：部署脚本先核对真实部署目录，冒烟项写死：首页、主文件、服务状态三项全过才算切换成功。	通道确认单	助理执行
7	**建立项目台账**：一张表管全部 50 个功能：编号、阶段、状态、验收人、验收日期。每周五更新一次。	项目台账	助理执行

> **第 1 周结束标准：**闸门 0 四项全勾，七件动作至少完成 1、2、7，其余有明确日期。达不到就顺延开工，不要带病启动 Phase 0。

第三章　分阶段操作卡：Phase 0-6

每张卡固定六栏：目标、你做什么、助理做什么、外部配合、产出物、过关标准。过关标准不满足，不进入下一阶段。

Phase 0　自动化基建 · 2 周 · F21、F22、F24

目标：让后面所有开发都跑在流水线上，告别手工部署。

你做什么：确认发布窗口与回滚授权；生产切换保留人工确认一步，不做全自动直发。

助理做什么：接通持续集成与自动部署；接入代码质量门禁；把多机运维写成脚本。

外部配合：无。

前置六项（先做）：一、Docker 环境安装——服务器装好 Docker 与 Compose，部署方式按第四章定案；二、代码生成器搭建——提前到本阶段（原排 Phase 2），437 个接口约六成是标准增删改查，生成器先行能省近一半编码时间；三、API 契约编写——核心模块的接口契约先写死，前后端按契约并行，不互相等；四、数据迁移调研——原系统表结构、脏数据、金额口径不一致现在就摸清，迁移脚本提前写、提前试跑；五、P0 安全项——全站加密连接、员工信息脱敏、强制改默认账号，搭环境时顺手做完，越往后改成本越高；六、Prometheus + Grafana 基础监控搭建——不等 Phase 3，本阶段就装好，开发期先看慢查询与内存问题。

产出物：可用的流水线、质量门禁规则、运维脚本、发布记录模板，以及容器运行环境、代码生成器、接口契约初稿、数据迁移调研记录、安全整改记录、dev-reset.sh 脚本、反向生成的 OpenAPI spec 初版、Prometheus + Grafana 基础监控面板。

过关标准：提交代码后 10 分钟内可在预发布环境验证；故意提交一段有缺陷的代码，流水线能拦住；一条命令完成部署且可回滚。

Phase 1　安全与断环 + 地基 · 6 周 · 关键路径

F2 数据权限　F3 工作流　F7 定时任务　F12 Excel　F13 对象存储　F19 认证　F25 接口测试　F27 回归测试　F37 进度条　F40 骨架屏

目标：先堵安全，再接断环。数据权限管住谁能看什么；工作流把审批、消息、到期提醒连起来；对象存储与流式表格处理打牢地基。

你做什么：确认数据权限三档口径（本部门、本部门及下级、本人）；确认先迁移的四个流程：合同审批、请假、报销、付款申请。

助理做什么：后端底座与登录链路先行；合同与证照到期提醒先做成定时任务；核心 20 个页面沉淀自动化回归；进度条与骨架屏随首期上线。

外部配合：财务确认金额字段规则；各部门确认数据可见范围。

产出物：可登录的自有后端、权限矩阵、工作流四件套、到期提醒、接口文档与回归集。

过关标准：分公司账号看不到其他公司的薪资；合同审批全程线上走通且历史可查；到期前 30 天和 7 天自动提醒；10 万行表格导入不崩溃。

> **关键路径提醒：**Phase 0（2 周）→ 后端底座（约 1 周）→ Phase 1（6 周）→ Phase 4（7 周）。这条线延期，全盘延期；其他阶段可以并行，关键路径不能插队。

第三章（续） 效率、可视化与体验

Phase 2 开发效率 · 2 周 · F1 代码生成器

目标：新模块不再从零手写。生成器本体已在 Phase 0 前置搭建，本阶段做的是模板完善与全量推广。

你做什么：确认生成代码的规范算不算数：响应格式、分页、权限注解必须与现有工程一致，不一致的生成器宁可不用。

助理做什么：配好单表、树表、主子表三套模板；做代码生成控制台。

外部配合：无。

产出物：生成器与使用说明。

过关标准：新建一张测试表，10 分钟生成前后端增删改查，零改动跑通。

Phase 3 可视化、文档与可观测 · 5 周

F5 报表设计器　F6 大屏　F11 在线预览　F15 全文检索　F23 监控告警　F29 日志集中　F32 图标统一　F38 图表主题

目标：报表不用等开发，大屏有趋势，文件不用下载，系统出事有人知道。

你做什么：点名先迁移的三张报表（项目进度、薪资、材料统计）；确认告警分级：谁收、什么级别打扰到什么程度。

助理做什么：报表与大屏先行；附件预览接对象存储；合同与制度做全文检索；监控、日志、告警一次接齐；图标与图表主题全站统一。

外部配合：业务部门试拖一张报表并反馈；告警接收人确认联系方式。

产出物：可自助的报表与大屏、预览入口、全文搜索、监控大盘与告警规则。

过关标准：业务人员能自己拖出一张新报表；搜关键词能定位到合同正文并高亮；停掉一个服务，5 分钟内收到告警；日志能按一次请求串起前后端。

第三章（续） 体验补齐与压测

Phase 4　体验补齐与压测 · 7 周 · 关键路径终点

F4 在线表单　F8 日志增强　F9 消息中心　F16 甘特图　F20 移动端　F30 压测　F31 打印　F33 富文本　F34 暗黑模式　F35 裁剪　F36 分片上传　F39 水印

目标：把每天用得到的体验补齐，并给出系统到底能扛多少人的答案。

你做什么：确认移动端首期只做三件事：审批、打卡、消息；确认哪些页面加水印（薪资、合同详情、投标报价）。

助理做什么：消息中心收口站内信与实时推送；项目页加甘特图；打印、富文本、暗黑、裁剪、分片上传逐项落地；每阶段末做一轮压测。

外部配合：一线人员试用移动端一周；财务试打一张付款单与发票套打。

产出物：消息中心、移动端轻量版、甘特图、打印模板、容量基线报告。

过关标准：审批流转全程有消息；手机能审批、打卡、收消息；排期拖拽可改、延期自动标红；给出明确容量基线，性能退化能自动拦住发布。

这一阶段的推进技巧：快慢分开排

类型	事项	排法
快赢项（1-2 天）	暗黑模式、水印、图片裁剪、富文本	穿插在重活之间上线，让使用者尽早看到变化，维持信心。
重活（1-2 周）	消息中心、移动端、甘特图、分片上传	单独排期、单独验收，不与快赢项混在同一张验收单里。
硬骨头	打印套打精度、离线与弱网体验	先拿一张真实单据、一个真实工地场景试点，达标再铺开。

压测不在最后才做：本阶段每个小版本收尾都跑一轮核心场景压测，结果记入台账，退化当周处理，不攒到上线前。

第三章（续） 集成预留与新赛道

Phase 5　架构预留与集成 · 随主线 / 按需

F10 多租户预留　F14 单点登录　F17 设备物联　F18 数据同步　F28 自动备份　F26 办公自动化

目标：便宜的预留现在做，贵的集成有条件再做。

你做什么：只拍板两件事：多租户字段预留直接做（成本低、以后贵）；单点登录、设备物联、数据同步，出现真实第二套系统或真实设备需求时再立项。

助理做什么：所有业务表预留租户字段且默认不启用；自动备份做成定时全量加增量、异地存放、每月恢复演练；办公自动化先试点发票录入与日报表两个场景。

外部配合：备份恢复演练需安排停机窗口；办公自动化试点需业务人员提供真实样本。

产出物：预留字段规范、备份与演练记录、两个自动化试点结果。

过关标准：模拟删库能从备份恢复且演练有记录；原来 2 小时的人工录入，自动化 10 分钟跑完且结果可核对。

Phase 6　新赛道 · 单独立项，不计入主线工期

档位	功能	立项条件
第一档 可直接立项	F45 规则引擎、F48 离线应用、F49 国际化（仅新模块与移动端）、F41 智能问答（先做制度问答试点）	F45 随后端引入；F48 必须等 HTTPS 上线；F41 需先整理制度文档并备好调用预算
第二档 过评估再立项	F42 自助分析、F43 协同编辑、F50 行为埋点	分别过运维人力、法务协议、隐私制度三道评估，评估结论存档后才排期
第三档 暂缓	F44 企业即时通讯、F46 自建视频会议、F47 对外电子签章（自建）	无公司级决心不做即时通讯；无合规硬要求不自建会议；对外签章只走持牌服务商

你做什么：每个新赛道单独听一次 15 分钟评估，只回答三个问题：解决谁的什么问题、不做会怎样、一年总成本多少。答不清，不立项。

过关标准：每个立项有独立验收单；试点先行，达标才推广，不达标就关停并写明原因。

> **排期纪律：**Phase 6 任何一项都不得挤占 Phase 0-5 的人力。新能力再亮眼，也不能让地基和断环延期。

阶段门怎么开

1. 助理提交产出物与证据，逐条对照过关标准自检。
2. 验收人按附录的阶段验收单打勾，缺项写明补交日期。
3. 你签字放行下一阶段；不放行就只做缺项，不开新工。

第四章　部署方案：Docker Compose 定案

部署方式已经定案：**用 Docker Compose**。理由很直接——本项目是单台服务器、十多个组件（数据库、缓存、文件存储、检索、监控、告警，以及后续的智能问答、自助分析、协同编辑等）的场景，这正是 Compose 的主场。Docker 官方文档对这个场景的判断是：

> "Compose works well for single-host deployments — it is simpler to operate than Kubernetes when you only have one server."（Compose 很适合单机部署；只有一台服务器时，它比 Kubernetes 简单得多。）

四个方案对比：为什么是它

方案	结论	依据
Docker Compose	**推荐采用**	单机多组件的标准答案；每个组件用官方镜像一行命令启动；版本写死在配置文件里，升级与回滚都是换一个版本号；与持续集成流水线天然衔接。
systemd + 二进制（现状）	不再继续	组件少时勉强可用，十多个组件逐个手工安装必然出现版本冲突与环境漂移。此前的白屏事故，本质就是这种手工模式的代价。
Kubernetes	不采用	它是为多机高可用设计的。单台服务器上用它，等于为不需要的能力付出十倍运维复杂度，杀鸡用牛刀。
Ansible + Docker 混合	**实际采用形态**	与 Compose 不是二选一，而是分工：Ansible 管服务器初始化与配置分发，Compose 管组件运行。手册正文说的定案，指的就是这套组合。

混合架构：每层放哪，已经分好

层	方案	说明
应用（后端 + 前端）	云服务器 + Docker Compose	内部系统负载稳定，包月服务器最便宜可控；云厂商的容器托管按量计费适合流量波动大的对外应用，本项目用不上。
数据库 MySQL	**自建 MySQL（默认）**	随 Compose 一起部署，数据用持久化卷存放，配合自动备份与每月恢复演练。
云数据库 TencentDB	备选（已定）	自带高可用、自动备份与任意时间点恢复，能替代自建备份的大半工作量。注意两点：活动价仅限首单，预算按续费价算；规格按峰值 1.5 倍一次选对。何时启用由拍板人另行决定，现阶段不切换。
无状态组件	云服务器 + Docker Compose	缓存、监控、检索、文件存储等一行命令启动，无需托管服务。
镜像仓库	腾讯云 TCR	流水线构建的镜像需要存放处，个人版免费额度够用，比自建镜像仓库省事。
扩展组件	云服务器 + Docker，按需加机器	智能问答、自助分析、协同编辑等先在现有服务器跑起来，资源不够再加机器，不预先采购。

第四章（续） 迁移路径与脚本升级

四步迁移：不停机，先易后难

步	动作	要点
1	**装 Docker**	服务器一次性安装 Docker 与 Compose，约半天内完成，先在非业务时段操作并留好回退办法。
2	**先迁无状态组件练手**	缓存、监控先迁。这类组件丢了重建即可，用它们把启动、日志、网络、持久化卷的操作练熟。
3	**再迁有状态组件**	数据库与文件存储最后迁，且**先备份、验备份、再迁移**。迁完逐项核对数据量与关键记录，核对不过不切换。
4	**自研应用容器化**	后端与前端打成镜像接进流水线：构建、推送、拉取、上线全程自动，发布记录随流水线留存。

一键部署脚本随之升级

> **旧做法：**脚本把文件同步到预发布目录，冒烟后切换。
> **新做法：**流水线构建镜像并推送到镜像仓库，服务器执行 **docker compose pull && up -d** 拉取新镜像并滚动更新。回滚就是把版本号换回上一个再执行一次同一条命令。
> 第七章交付规范里的「预发布 → 冒烟 → 切换 → 可回滚」不变，变的只是切换的载体：从文件目录换成镜像版本，更干净，也不会再出现新旧文件混装。

诚实说明：三个局限，先讲清

- **要学几个新概念：**持久化卷、容器网络、容器日志怎么看。不难，但要留半天专门练，别在上线当天第一次碰。
- **单机仍是单点故障：**容器化解决的是部署与一致性，解决不了「机器挂了怎么办」。真要高可用得加机器，那是主线之后的事，现在不用想。
- **性能开销 1-3%：**容器有一层很薄的开销，对本项目的负载量级可以忽略，不必为此纠结。

> **本章结论：**部署定案 Docker Compose（Ansible 管分发、Compose 管运行）；数据库默认自建 MySQL，TencentDB 只作备选；迁移按四步走，第三步先备份再迁是硬规矩。

第五章　项目加速八法

最快的加速不是加人赶工，而是砍掉返工和等待。编码时间挤不动，硬挤只会挤出下一次白屏事故。下面八条，是本项目已经定下来的加速打法，排期与执行都按它来。

#	做法	怎么做、为什么快
1	**模块化单体，千万别碰微服务**	后端是一个 Spring Boot 应用，按业务模块划分代码包（项目、合同、财务、人事……），模块之间只通过接口调用。行业共识是「大多数应用都该从模块化单体起步」，微服务的基础设施成本约为单体的 10 倍。以后真要拆分，模块边界就是现成的拆分线。
2	**代码生成器提前到 Phase 0**	437 个接口里约六成是标准增删改查。生成器先行，后面每个模块都是「配表结构 → 生成 → 微调」，能省近一半编码时间。这是投入产出比最高的提前量，已写入 Phase 0 前置六项。
3	**API 契约先行**	核心模块的接口契约先写死：前端按契约对接模拟数据，后端按契约实现，两边并行、互不等待。三线并行能成立，前提就是这一条。
4	**渐进式替换（绞杀者模式）**	新后端先跑通登录、项目、合同三个核心模块，老系统继续并行运行，其余模块逐个迁移、逐个验收，随时可切回。行业提醒很直白："Rushing a big-bang rewrite is how projects die."（赶工式一次性重写，是项目死亡的方式。）
5	**数据迁移在 Phase 0 就启动调研**	原系统数据存在表结构不一、脏数据、金额口径对不上等问题。迁移脚本提前写、提前试跑、提前对账——数据迁移不是收尾阶段的事，拖到最后必然延期。
6	**特性开关兜底**	新功能默认关闭，按部门、按项目灰度放开。出问题关开关即可，不用回滚整个版本。这是「稳定完成」四个字的硬保障。
7	**每周五 15 分钟演示**	每周五由执行方演示本周成果，拍板人看效果、提意见。方向走偏一周就能发现并纠正，而不是三个月后验收时才发现。这是性价比最高的防返工机制。
8	**P0 安全项塞进 Phase 0**	全站加密连接、员工信息脱敏、强制改默认账号，搭环境时顺手做完。安全项越往后改成本越高，已写入 Phase 0 前置六项。

> **两处不能快（红线）：**一、后端接口的工作量是实打实的，不能靠压缩工期省出来，硬压就是埋雷；二、拍板人的验收时间不能省——1 小时验收，能省掉执行方 3 天返工。

第五章（续） 效率工具四件

八法解决的是打法，四件解决的是工具。这四件都来自 2026-10-19 Docker 验证之后的复盘：手头已经有现成资产，就不要再手写一遍；能自动对比的，就不要靠人眼验收。

1. API 契约反向生成：不用手写契约

复刻站已有 **44 个 API 模块、760 个接口函数**，本身就是现成的契约。写一个脚本，从前端 API 定义反向生成 **OpenAPI spec**，后端照着实现即可。

- **收益：**第五章第 3 法「契约先行」的落地成本砍掉约 **80%**，不用逐个接口手写文档。
- **做法：**Phase 0 产出 spec 初版，人工只复核命名、分页与错误码口径；spec 进仓库，后续变更先改 spec 再改代码。

2. 复刻站做后端的「对照组」：有标准答案就用

后端每实现一个接口，自动对比「原站返回 vs 新后端返回」的差异（diff），不一致就报警，不靠人工逐页点验。

- **收益：**1:1 复刻最大的优势就是有标准答案，不用这个等于浪费；字段缺失、口径偏差在联调阶段就暴露。
- **做法：**对比时先脱敏，再比结构、状态码与关键字段；金额、日期类字段单独设口径规则，diff 报告随验收单归档。

3. 开发环境一键重置：dev-reset.sh

一条脚本走完三步：**重建数据库 → 跑 Flyway 迁移 → 导入基础数据**，5 分钟得到一个干净环境。

- **收益：**加人、新开分支测试、复现问题时不再手工搭环境；「我这儿能跑、你那儿不行」的环境差异直接消灭。
- **做法：**脚本随 Phase 0 交付并进仓库；重置只允许在开发与预发布环境执行，生产环境禁用并加确认门槛。

4. 监控 Phase 0 就装上：不等 Phase 3

Prometheus + Grafana 从原定的 Phase 3 提前到 Phase 0（已列入前置六项第 6 项）。开发阶段就能看到慢查询和内存泄漏。

- **收益：**问题在开发期发现，成本是生产期的 **1/10**；容量与性能基线从第一天开始积累，不是上线前才测。
- **做法：**先接最基础的一套：主机与容器资源、后端响应耗时、数据库慢查询；告警先只报不吵，阈值在 Phase 3 再按分级收紧。

> **四件的共同原则：**先用现成资产（复刻站、API 定义、原站返回），再写自动化脚本，最后才靠人工。顺序反过来，就是把已经付过的成本再付一遍。

第六章　通用执行 SOP：每个功能的 8 步

50 个功能看起来很多，做法只有一种。任何一个 F 编号，从立项到归档都走这 8 步，跳步就是埋雷。

步	动作	怎么做才算对
1	**立项卡**	一页纸写清：解决什么缺口、谁受益、验收标准、预计工期、不做的影响。写不清就不开发。
2	**设计确认**	接口先有文档，页面先有样子，数据先定字段与必填规则。涉及金额、权限、隐私的设计，必须过对应外部确认。
3	**开发**	在复刻工程与自有后端开发；新表优先用生成器起步；规则、文案、配置不硬编码到无法修改的位置。
4	**自测**	开发者自测加自动化：接口用例、核心页面回归、代码质量门禁，三项全绿才提部署。
5	**部署预发布**	走唯一通道先到预发布环境，冒烟通过后再申请生产切换。禁止直接覆盖线上。
6	**真机验证**	在服务器上用真实浏览器走一遍真实流程，留截图或录屏证据。本地通过不算数。
7	**验收签字**	验收人按立项卡逐条打勾，填附录单功能验收单。你抽查高风险项，其余信任验收单。
8	**归档同步**	代码进仓库、文档进知识库、台账改状态、部署有记录。四处对齐才关单。

最常见的三种跳步：没立项卡就开发（范围失控）、本地测过就上线（线上翻车）、上线了不归档（下次无人知道改过什么）。见到任何一种，直接打回上一步。

第七章　交付规范：唯一通道与验收证据

环节	硬性要求
代码检查	格式检查、类型检查、构建，三项全部通过才允许部署。任一红灯，停。
发布通道	只走一键部署脚本：先传预发布目录，冒烟通过后瞬间切换，保留最近两个版本可回滚；容器化落地后按第四章改为拉取镜像滚动更新，预发布、冒烟、切换、可回滚的流程不变。禁止手工同步直接覆盖线上目录，这是出过白屏事故的根因。
冒烟清单	首页正常打开、主要文件正常加载、服务处于运行状态。三项写进脚本自动执行，结果留记录。
真机验证	部署后在服务器侧用真实浏览器验证核心流程，并给出证据。验证不过，对外口径只能是「已部署、未通过」，不能说「已修复」。
回滚	切换后发现异常，优先回滚到上一版本再排查，不在生产环境现场改代码。
同步	工作区与服务器、文档与仓库保持同步；提交信息用规范前缀，写清改了什么。

第八章　周节奏与沟通模板

时间	固定动作	产出
周一	**排期会（30 分钟）**：确认本周要交付的功能编号、验收人和风险。	本周任务表
每天	**异步同步（5 分钟）**：昨天完成、今天要做、卡在哪里。卡住超过一天必须升级给你。	三行日报
周三	**中期检查**：看预发布环境，不看口头进度。能演示的才算进度。	演示记录
周五	**演示 + 验收会**：先用 15 分钟演示本周成果（见第五章第 7 法），再逐条过验收单，更新台账，确认下周闸门。	演示记录 + 验收单 + 台账

沟通模板（直接套用）

> **派单：**「功能编号 ___，目标 ___，验收标准见立项卡，完成日期 ___，验收人 ___。」
> **汇报：**「功能编号 ___，状态：已部署 / 已验证 / 已验收，证据链接 ___，风险 ___。」
> **卡点升级：**「功能编号 ___，卡在 ___，需要你拍板 ___，最晚 ___ 前，不定的影响是 ___。」
> **拍板回复：**「按默认走」或「改第 ___ 条为 ___」，一句话即可，助理负责落记录。

你每周只需要看三样东西

1. **台账红黄绿：**红色项逐个问原因，黄色项问日期，绿色项不细看。
2. **验收证据：**抽查 1-2 个高风险功能，看真机验证记录。
3. **下周闸门：**下周开工条件是否齐备，缺什么、谁补、何时补。

第九章　角色分工表

事项	拍板	执行	配合 / 验收
范围、预算、暂缓与关停	你	助理整理选项	—
金额口径与字段必填规则	你确认	助理落地	财务确认口径
数据可见范围	你确认	助理实现	各部门负责人
开发、部署、真机验证	—	助理	验收人抽查
报表、大屏、打印模板试用	—	助理搭建	业务人员试用
签章、协议、隐私类立项	你	助理整理材料	法务 / 人力先评估
生产切换与回滚	你授权	助理执行	—

原则：一件事只有一个拍板人。表里没写的事，先问「这事谁拍板」，定完人再开工。

第十章　决策清单：9 项已拍板（2026-10-19）

> **状态：已决策、未开工，等开工令。**九项决策已于 2026-10-19 由拍板人确认，全部按默认项通过。本章不再是待填清单，而是开工依据：任何人对范围、技术与顺序有疑问，先查这一页。

A. 后端四项（已通过）

事项	已拍板结论	状态
1 技术栈	**Spring Boot 3.2 + JDK 17。**原系统实测就是这套，接口行为对齐成本最低，50 个功能里多数开源件也是这个生态。	已通过
2 数据库	**MySQL 8.0（自建）。**随 Docker Compose 部署；云数据库 TencentDB 定为备选（见第四章），不作为当前默认。	已通过
3 启动范围	**Phase 0+1 先行。**先交付「登录可用」验证整条链路，再全量开工，反馈周期最短。	已通过
4 部署位置	**同机部署。**与复刻站同一台服务器，采用 Docker Compose（第四章）；数据量变大后再迁独立机器。	已通过

B. 工程五项（已通过）

事项	已拍板结论	状态
5 一键部署脚本	**采用。**采用前先确认服务器真实部署目录；容器化后脚本升级为拉取镜像并滚动更新（第四章）。	已通过
6 GitHub 公开版历史重写	**暂缓。**当前公开内容已脱敏，重写历史不可逆、成本高，出现合规要求再处理。	已通过
7 OCR 接入复刻站	**接入。**发票识别接入复刻站发票模块，演示版本已验证识别效果，随主线推进落地。	已通过
8 补跑复查 82-170 页	**暂缓。**已有结论够支撑决策，补跑只为完整性，不挡主线，有空再补。	已通过
9 推进方式	**三线并行。**后端、前端美化、运维基建三线同时推进，主线工期约 14-16 周；每周五固定 15 分钟决策与演示，批量清待定事项。	已通过

用户侧待办（不算决策）：DeepSeek 充值。充完通知执行方即可恢复默认编码路线；未充前用现有替代路线顶着，不挡开工。

拍板日期：2026-10-19　拍板方式：逐项勾选默认项确认　记录状态：已同步进项目台账口径

> **开工令规则：**本章结论已经齐备，项目启动只差一句开工令。开工令下达前，不启动 Phase 0；下达后 24 小时内，执行方把本章结论逐项落进台账与立项卡。

第十一章　风险红线：触碰即停

红线不是风险提示，是停工条件。碰到任何一条，先停、先报告、先拿到书面结论，再谈继续。

红线	为什么	正确做法
对外合同用自建签章	可靠电子签名需要认证机构背书，自建签章在纠纷时可能不被认可，合同等于没签。	对外走持牌服务商；自建只用于内部流程留痕。
协同编辑的开源协议	所选方案带传染性开源协议，改代码对外提供服务可能触发开源义务。	法务先确认合规边界，结论存档后才部署。
员工行为埋点无告知	行为数据涉及个人信息，无告知、无制度就采集，合规与信任双输。	只统计功能热度不追踪个人；上线前发告知并立制度，人力与法务确认。
手工覆盖线上部署	新旧文件混装出过白屏事故，属于已验证会翻车的做法。	只走唯一通道：预发布、冒烟、切换、可回滚。
未上 HTTPS 就做离线应用	离线应用技术上强制要求加密连接，条件不满足做不了，硬做是白工。	先完成 HTTPS，再立项离线应用。
靠热情推全员换沟通工具	即时通讯与视频会议的最大成本是改习惯与持续运维，不是软件本身。	无公司级决心就用现成服务；自建只在有合规硬要求时考虑。

一般风险与对策（不用停工，但要有人盯）

- **工作流引擎学习曲线：**先迁四个简单流程练手，复杂会签放二期。
- **数据权限改造贵：**先做薪资、合同、员工档案三个高风险模块，其余用到再加。
- **自动化测试脚本易碎：**页面元素用稳定标识定位，失败自动截图，改版同步改脚本。
- **压测打挂环境：**只在预发布环境压，生产压测需另行授权并限流。
- **告警风暴：**告警分三级并去重，半夜只叫醒真正紧急的那一级。
- **智能问答答错：**试点期人工复核，准确率达标才推广；合同审核类高风险场景暂不碰。
- **原后端不稳定传导：**原系统接口间歇报错会影响复刻站数据，监控要能分清是自有问题还是上游问题。

附录 A　项目启动清单（复印填写）

☐ 闸门 0 四项已确认（G1 地基 / G2 通道 / G3 后端四项 / G4 验收人）

☐ 第十章决策清单已确认（2026-10-19 九项已拍板），结论已同步进台账

☐ 项目台账已建立，50 个功能全部登记

☐ 发票识别定性结论已归档（接真 / 禁用保存）

☐ 金额字段规则表已由财务确认

☐ 数据可见范围已由各部门确认

☐ 唯一发布通道已核对部署目录并试跑一次

☐ 周节奏已定：排期会、验收会的固定时间已进日历

☐ 红线清单已发给全体执行人员并确认知晓

启动日期：_____　拍板人：_____　记录人：_____

附录 A-1　第一周派单记录表

序号	事项（对应第二章）	负责人	完成日期	已勾
1	放行后端四项默认			☐
2	发票识别定性			☐
3	启动 HTTPS 排期			☐
4	清理默认账号风险			☐
5	冻结金额必填规则			☐
6	确认唯一发布通道			☐
7	建立项目台账			☐

填法：周一派单时填负责人，周五验收会逐行勾选并填日期。未完成的行自动滚入下周，并在台账中标黄；连续两周未完成标红并升级拍板。

附录 B　阶段验收单（每阶段一张）

阶段	Phase _____ 名称：_____
验收期间	自 _____ 至 _____
过关标准核对	☐ 全部达标　☐ 部分达标，缺项：_____ 缺项补交日期：_____
证据	☐ 真机验证记录　☐ 测试报告　☐ 监控 / 日志截图　☐ 试用反馈
结论	☐ 放行下一阶段　☐ 只补缺项，不开新工　☐ 暂缓，原因：_____
签字	验收人：_____ 拍板人：_____ 日期：_____

附录 C　单功能验收单（每个 F 编号一张）

功能编号	F _____ 名称：_____ 所属阶段：Phase _____
立项卡标准	逐条抄录立项卡验收标准，并逐条打勾： ☐ 标准 1：_____ ☐ 标准 2：_____ ☐ 标准 3：_____
交付检查	☐ 代码检查三项全绿　☐ 走唯一通道部署　☐ 冒烟三项通过　☐ 服务器真机验证通过
归档检查	☐ 代码进仓库　☐ 文档已更新　☐ 台账状态已改　☐ 部署记录完整
结论	☐ 通过，关单　☐ 打回，原因：_____
签字	执行：_____ 验收：_____ 日期：_____

> **手册到此结束，项目从这里开始。**九项决策已经拍完，下一次沟通只需要一句话：开工。剩下的，按周推进、按单验收、按红线刹车。

附录 D　Docker 验证结论备忘

记录日期：2026-10-19　验证对象：现有服务器（4 核 4G）上的 Docker Compose 核心栈　性质：开工前环境验证，非业务开工

实测结果

项目	实测
MySQL 8.0	内存稳定在 **465MB**（上限 1.5G），启动后健康检查通过，读写正常。
Redis 7	内存仅 **4MB**，读写正常。
整机余量	两个容器运行期间整机仍余约 **1.5G 可用**，够再承载一个后端服务与前端。
镜像拉取	核心镜像 **35 秒**拉取完成，服务器到镜像仓库的网络无障碍。
对现有业务的影响	验证全程复刻站服务保持运行、首页正常返回，**未受任何影响**；验证结束后容器与数据卷已清理干净。

> **结论：**现有服务器可直接承载 Phase 0 核心栈，**无需先买新机器**即可开工。新购 4 核 16G 开发服务器推迟到 **Phase 3 前后**，待检索、监控等重组件上齐时再按需投入；届时以本备忘的实测水位为选型依据，不凭感觉采购。

使用说明：Phase 0 开工时按此结论直接在现有服务器搭核心栈，并给数据库与后端设好内存上限；若后续实测水位与本备忘明显不符，先停下复核，不要直接加购机器掩盖问题。